



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

TRINITY COLLEGE DUBLIN

EEU22E10

Engineering Design IV Final Report

Author

Argjenda Celina	24372200
Cathal O'Sullivan	23374706
Eoghan Taylor	23374361
Henry Olenich	23347869

April 13, 2025

Contents

1	Introduction	2
2	Part A: Self Assessment	2
2.1	Strengths of the Team	2
2.2	Weaknesses of the Team	3
2.3	Buggy Improvements	3
2.4	Comments	4
3	Part B: Hardware Design	5
3.1	Block Diagrams	5
3.2	Fritzing Diagrams	7
3.3	Hardware Challenges	8
4	Part C: Software Design	9
4.1	Overview	9
4.2	WiFi Transmission	9
4.3	Processing	10
4.3.1	Inputs from user	10
4.3.2	Data Transmission	10
4.3.3	GUI	11
4.4	PID	11
4.5	Object Following	12
5	Part D: Gold Challenge	13
5.1	Overview	13
5.2	HuskyLens Code	13
5.3	Data Transmission	13
5.4	Updates to Processing	14
5.4.1	Decoding	14
5.4.2	GUI	15
6	Part E: Change Log	16
7	Conclusion	16

1 Introduction

Over the course of this semester, a task was assigned to design, wire, and program a buggy to follow a track autonomously. The specific requirements included having the buggy follow a white line on a black backdrop, communicate with a control PC, stop in front of objects, maintain a specified speed using a PID system, follow an object in front of it at a safe distance, and recognize April tags using a camera causing it to carry out certain predetermined acts. There were 3 challenges:

- **Bronze Challenge:** Deadline Week 6: The challenge consisted of following a track, stopping for obstructions, and communicating with the control PC. The group completed this task on time.
- **Silver Challenge:** Deadline Week 9: The challenge consisted of implementing a PID system to control velocity, as well as a mode for the buggy to follow objects in front of it. The group completed this task on time.
- **Gold Challenge:** Deadline Week 12: The challenge consisted of implementing a system for the buggy to recognize a set of April tags which would cause the buggy to carry out certain functions. The group was able to implement recognition of the April tags, but while demonstrating, the buggy veered off track and so the group could not finish the gold challenge.

2 Part A: Self Assessment

2.1 Strengths of the Team

The team exhibited strength throughout the course of the 12 week project and one of the top strengths was active communication. From the beginning of January, a WhatsApp group chat was used to organize team meetings, stay on top of tasks, and ensure that everyone was consistently on the same page. Each team member was able to use the chat to cover their own needs and help the team move forward by asking questions, bringing up any issues, or simply informing the group with an update on the buggy code.

Furthermore, organized delegation of work throughout the past few months was paramount to the group's success and was also a top strength. This primarily occurred during in person meetings when there was either something to submit or a task to check off. For example, when the interim report was assigned, each group member shared their preference in what sections they wanted to contribute most towards. Within a few minutes, each member had a section 'assigned' to them to work on for the next week or two. For each task and assignment, the group reached an agreement on workload distribution in minutes, which means that peak efficiency was achieved in work delegation.

A third strength was the team's ability to problem solve and troubleshoot together. This would occur primarily during lab sessions on Mondays and Wednesdays, and the team encountered an issue with the buggy. For example, after a few runs around the track, the buggy would occasionally lose power and barely be able to move forward. The team worked together on this issue by troubleshooting different parts of the project. Someone would take a look at the hardware components and someone would look at the buggy code. If it was guaranteed that we could cross something off the list, then the group would focus on whatever

could possibly be causing the problem. In this case, we decided it was not the code, so we replaced the batteries with brand new ones. After it was determined that it might not be a battery issue, we replaced one of the motors and it seemed to solve the issue. Our team's collaboration and step by step thinking during troubleshooting helped our buggy succeed.

2.2 Weaknesses of the Team

One of the weaknesses of the team was that the understanding of the buggy was at times stratified. Members would often be working on different things and so for example one member might understand more about the H-bridge than the others while someone else would understand more about implementing the Wi-Fi. This weakness was difficult to avoid because it is simply hard for two people to be working on the H-bridge implementation at one time. The team worked on solving the difference in understanding by teaching the other members how their task worked and by having the other members try to make it work themselves.

Another weakness of the team was that we had a slight fear of conflict. Throughout the project, the team ran very smoothly and so this could be seen as both a strength and a weakness. The weakness side to this is that when a member would do something or have an idea, for the most part, we would try to implement it or let it happen. That happened to work out for this group because our personalities meshed well together but the avoidance of conflict meant that we did not discuss changes or new ideas to the full extent.

A final weakness of the team was that we did not set clear individual expectations. As a team, we set expectations and goals so that work would get done but individually, we did not have expectations for each other. This resulted in especially code work occasionally being done by an individual. Of course, it is difficult to work on code with more than one person at a time but even so, if someone was passionate about getting some code done, they would do it. This is a weakness because individual expectations could have helped prevent the occasional work being done by one person by making it more clear about what people wanted from the project from themselves and from the others.

2.3 Buggy Improvements

Given more time to work and tweak the buggy, there are a few specific changes that the group would have made to help guarantee the achievement of the gold challenge.

First, the PID would have been tweaked to change speeds less rapidly. In our gold demo, the buggy changed speeds very quickly which caused it to run off the track a few times. This was likely the main reason why the buggy was unable to complete the final challenge loop. If the buggy velocity increased and decreased less rapidly, then it would be easier for the sensors to react to any sudden changes and keep the buggy on the track.

Another change was something that also might have contributed to the buggy running off the track - the IR sensors. The left IR sensor seemed to have increased sensitivity than the one on the right and as a result the buggy would most often run off track to the left. The group believes that it might be attributed to glare on the black mat and so the left sensor picked up white, causing the buggy to turn left and neglecting the right sensor because it was less sensitive than the left one.

Finally, the setup of the QR codes on the track would have made the final track challenge run smoother. The team's QR codes were just printed pieces of paper with a fold in them to help it stand up on the side of the track. It would be an improved design to attach these QR codes to a more sturdy structure such as cardboard to hold them up. In the gold demo, group members had to physically hold up the codes because they kept falling down.

2.4 Comments

There are a few points about ethics that are relevant to the buggy project and autonomous vehicles in general. One of the most important points brought up in this sector has to do with the data that the buggy might collect. This buggy specifically did not collect data having to do with humans - it was mainly just based on sensor readings of objects and surfaces. However, more advanced autonomous vehicles might be able to capture human's voices for directions or track human uses which could then be questioned in terms of ethics.

When working on anything that requires electricity and power, the question of energy and sustainability comes into play. This was a 12 week project that required constant charging of batteries and computers. It required us to replace the motor and the hall sensor as well. On one hand, the buggy was used as were all of its parts which is a sustainable practice. On the other hand, electricity was used to charge the batteries and computers that were used during the project. Not a significant amount of electricity was used solely for this project so it is more important to bring up the significance of electricity use rather than propose a solution.

Another topic on ethics is using other people's information to work and learn for the project. Most of the information was from the slides which was provided for team members by the lecturers or on online public forums provided by strangers, expecting it to be used by others. This means that there is not much ethical conflict to debate about in terms of using information to lead us to solutions.

A final comment on the ethics of the project has to do with the outputs of the project. For some projects, the team members may have to consider if the outputs of their research will be used for good or not. In this particular project, the output was on a more local and individual level, primarily the gained knowledge of electronics for the team members. The outputs were the creation of the buggy hardware which will be taken apart and put back together next year and the software which was used to run the buggy. The outputs were more on a conceptual level of what the team members learned rather than a research paper, dataset, or some solution to a world issue.

3 Part B: Hardware Design

3.1 Block Diagrams

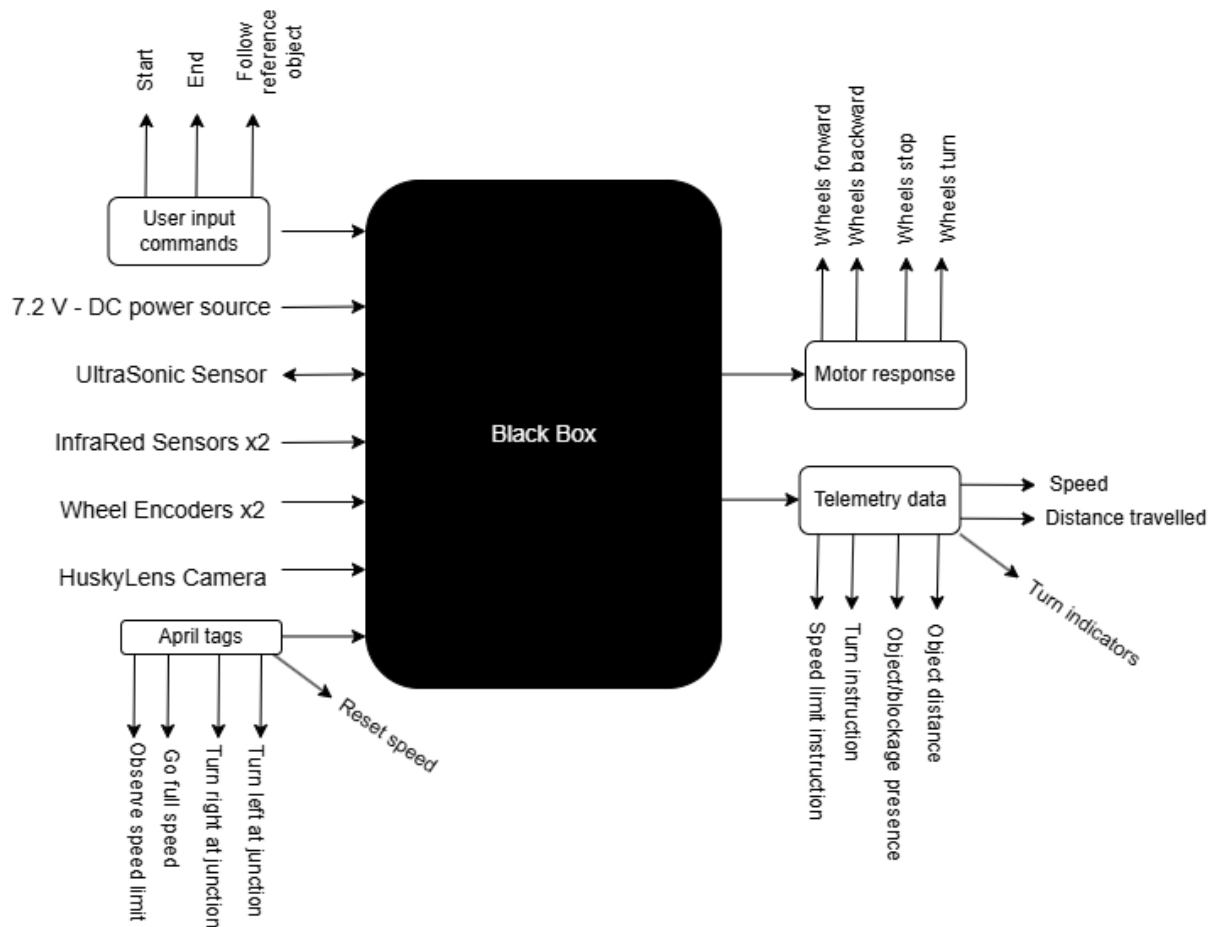


Figure 1. *Level 0 Block Diagram*

As seen in Figure 1, we've listed the input factors on the left, keeping in mind that the US sensor needs a continuous Trigger/Echo communication to function thus it is both an input and output component. In addition to the block diagram we had in the interim report, the HuskyLens Camera has been added. It takes inputs by observing and analysing the April tags, which may instruct to turn, slow down, go at the maximum speed that the buggy can safely operate at or reset by removing any speed conditions.

Thereafter, on the right we've listed the outputs of our system, hence meaning the reaction and response to the inputs that our buggy carries out, along with the data reporting it sends back to the user representing the progress of those actions (distance travelled, speed, whether an obstacle has been introduced right in front of the buggy, turn or speed instructions from April tags, etc).

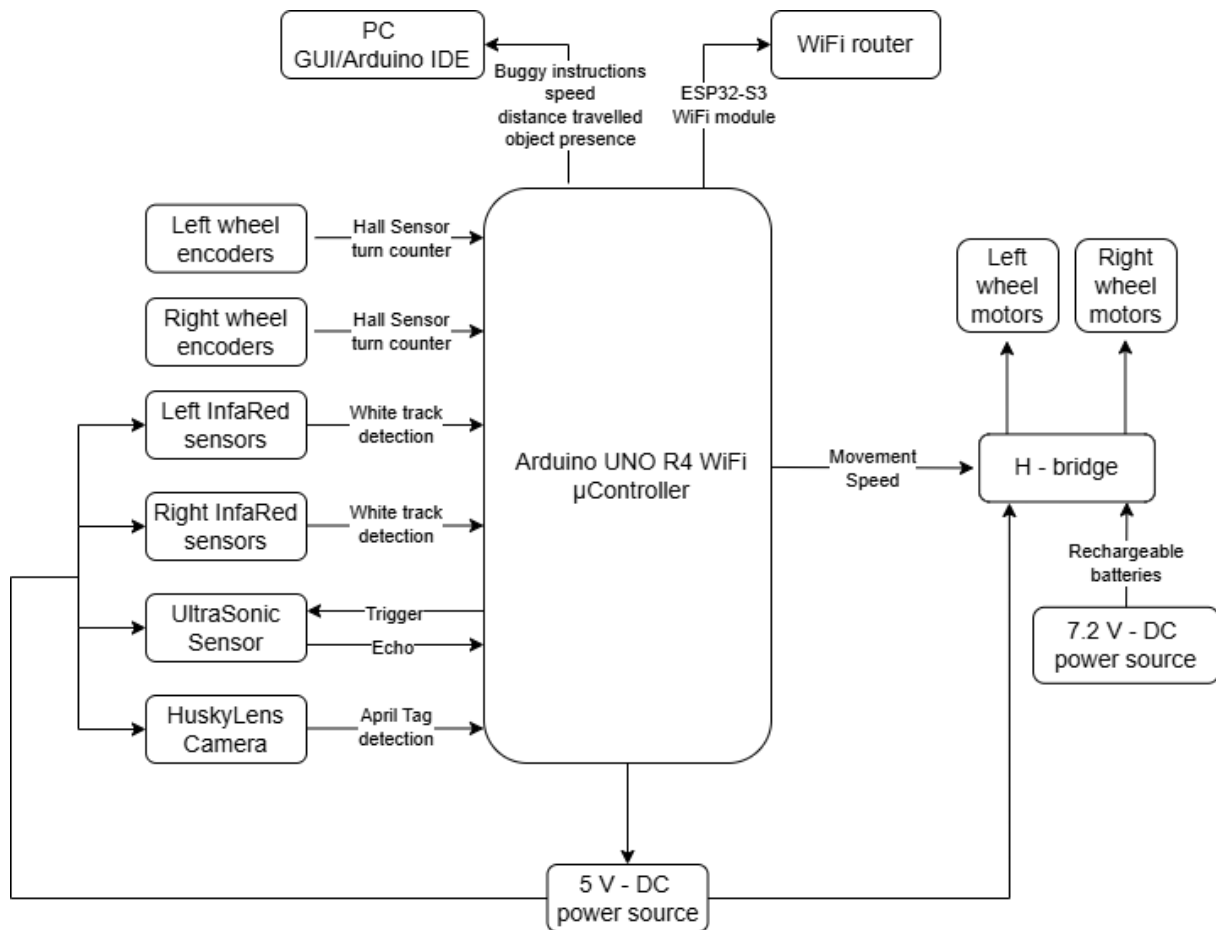


Figure 2. Level 1 Block Diagram

In this next level of the block diagram, we observe more detailed and elaborate connections explaining how the sensors, user inputs, Wi-Fi communications, power sources and hardware components interact with each other.

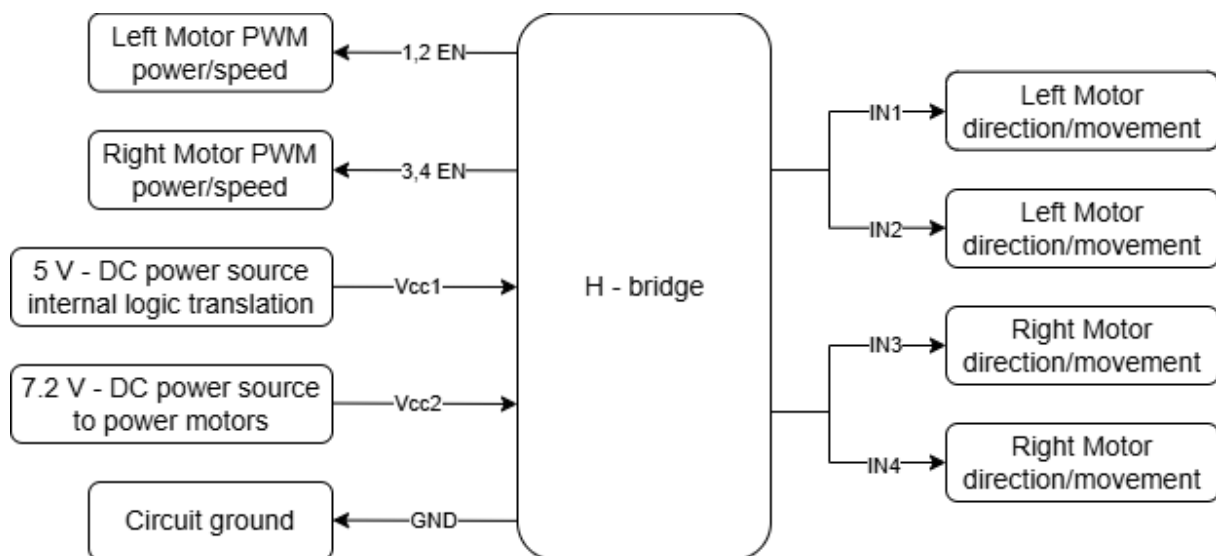


Figure 3. Level 2 Block Diagram, H-Bridge detailed

Considering the H-bridge is one of the more complicated components in our circuit, we decided to dedicate a Level 2 Block diagram to it to better understand its connections. It's different pins have different purposes such as whether the wheels turn forward, back or do a full stop, and finally the speed at which they move. For these functions to be possible we need to connect the motors to a 7.2V power source provided by the batteries plugged into the buggy, and a 5V power source which serves to power the internal logic translation.

3.2 Fritzing Diagrams

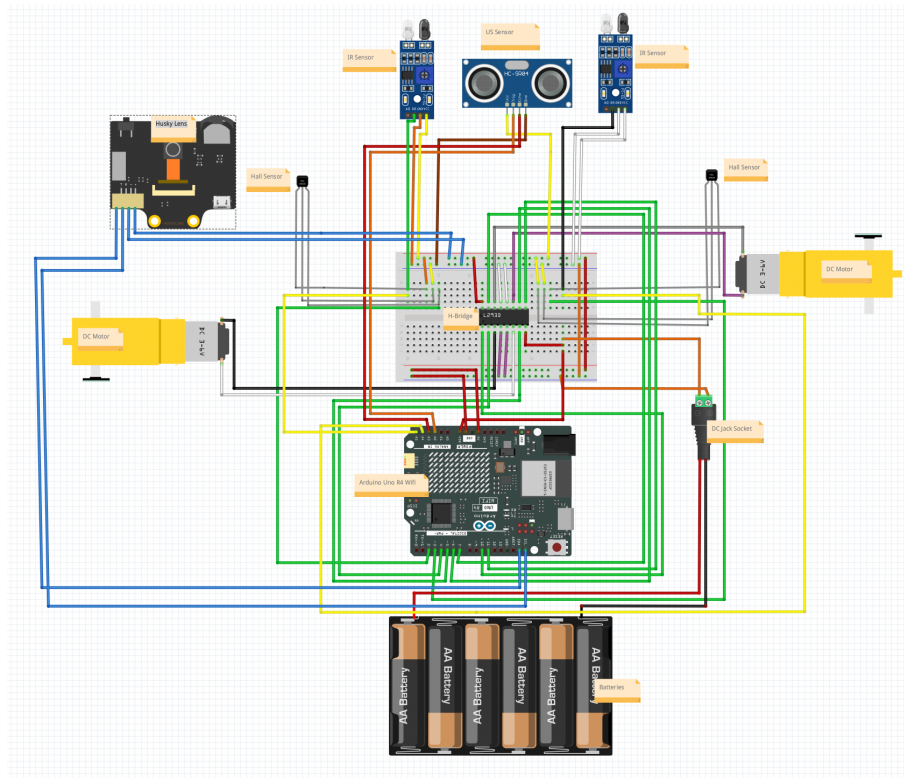


Figure 4. Fritzing diagram of the breadboard, devices, and wiring associated with it

The breadboard layout pictured below depicts our group's wiring on the buggy which can also be represented by the circuit schematic. The space of the breadboard was used efficiently with a layout that is simple to read and understand. It is nearly symmetrical, with the H-bridge positioned as a center piece. The 5V from the Arduino is wired to rails on both sides of the breadboard so that wires can be easily spread out and organized rather than clumped up on one side. The H-bridge is used to control the motor direction and speed using pulse width modulation pins from the Arduino. The IR sensors and US sensor are connected to analog pins in the Arduino because the physical wires did not reach the digital pins and in terms of organization it made it simpler. The IR sensors function to keep the buggy on track and the US sensor functions to stop the buggy in case of obstacles in the way. The IR sensors emit infrared radiation, and the US sensor sends ultrasonic waves out and measures the time it takes for it to bounce back to the buggy. The camera is also connected to the Arduino through the SDA and SCL pins, while powered through the 5V rail this device is used to scan and read April Tags set up on the track, serving to give signals to the buggy to give an instruction or a constraint (turn instruction or speed limitation). The batteries are connected to the h-bridge which powers the DC motors, however, it cannot connect to the Arduino or else damage might occur.

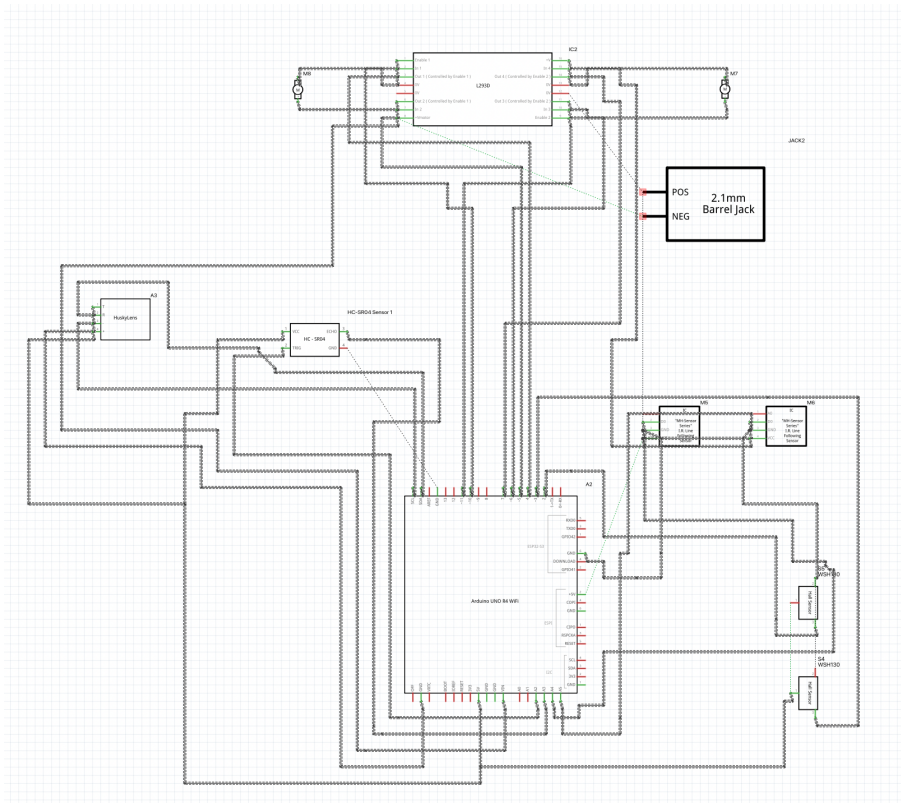


Figure 5. Fritzing diagram of the Circuit Schematic

Finally, the wheel encoders send signals to the Arduino when the wheel rotates using a magnet that emits a pulse when another magnet on the wheel passes by on its rotation.

3.3 Hardware Challenges

There were a couple of challenges that our group faced when tackling the wiring of the buggy. First, it is physically challenging for multiple people to wire a buggy at the same time which makes collaboration difficult. However, with harder wiring tasks, we would take a look at the buggy together and figure out where to connect certain wires. Sometimes someone in the group would attempt to wire the buggy and then other people would take a look to either fix it or learn it. Another challenge was that often our motors would not start without angling the wires perfectly. Every time we wanted to test something we would have to hold the wires in place with our hands and hope that it would stay in position. To fix this and also overcome the obstacle of the left wheel encoder not working, we replaced both DC motors with new ones.

Throughout the majority of our experiments and work, we struggled with the life span of the batteries. Using the 6 AA 1.2V rechargeable batteries initially provided, we came to discover that they would drain out within minutes of usage. We ended up buying new ones, establishing that the old ones had been worn and torn, however even then low power issues occasionally arose. This we believe is due to the high power demand of the buggy, and rechargeable batteries usually draining out faster than single use ones.

The final challenge we encountered was connecting the HuskyLens camera. It would rarely connect but without any changes to the code, leading us to believe that we had a hardware issue. The issue involved the A4 and A5 pins on the Arduino. These pins are hardwired

to the I2C bus, which is a serial communication protocol that the HuskyLens camera relies on, meaning it uses these same pins (A4 for SDA and A5 for SCL) to transmit and receive data.

However, we had previously connected the IR sensors to these pins, which created interference on the I2C bus. As a result, the Arduino could not establish a stable connection with the camera, leading to frequent failures. We resolved the issue by simply changing the pins of the IR sensors, thus allowing the I2C communication to function properly and the camera to connect without issues.

4 Part C: Software Design

4.1 Overview

The Arduino control loop repeatedly iterates through every function that the buggy needs to carry out, though some functions it carries out every loop, while others it only carries out every 400 to 600 milliseconds. These are the tasks carried out:

- Checks Wi-Fi connection: Every loop: Every loop the Arduino checks the Wi-Fi connection and if the connection fails it attempts to reconnect.
- Adjusts speed: Every 400 milliseconds: The buggy uses the wheel encoder to measure how far the buggy has travelled, and calculating an estimated velocity. Then, using the PID system outlined in a later section, it updates the voltage to maintain the correct target speed.
- Checks IR sensors for steering: Every loop: The Arduino takes in the digital inputs from the IR sensors, using them to steer in the direction of the white line.
- Checks for object using US sensors: Every 600 milliseconds: The Arduino activates the US sensors to send out a signal, using the times taken for the signal to reflect back to judge the distance to an object in front of it.
- Transmits messages with processing program: Every 600 milliseconds: The Arduino and the controlling computer connect and transmit data. More details outlined in subsequent section.

4.2 WiFi Transmission

The Arduino and the processing component were connected via WiFi connection. Both would connect to a common network set up in the lab and the Arduino would act as a server in this model while the processing component acts as a client.

Every 600 milliseconds in the Arduino program the Arduino attempts to connect the the client. The Arduino first tries to receive a signal composed of one float value. This float represents speed can be any float between 0 and 15, and it can also be -1. If the value is -1, this means that the computer is telling the buggy to follow an object in front of it (Outlined clearly later).

The Arduino then attempts to transmit its own message, consisting of 3 parts, separated by spaces and terminated by a new line. These parts are distance, voltage, and signals. Distance is transmitted as an integer value of the total number of centimetres travelled, and voltage

is transmitted as an integer from 0-255 representing voltage from 0-5V. Signals consists of 3 pieces of information, encoded into 1 integer. The 3 pieces of information are:

- Is the buggy turning left?
- Is the buggy turning right?
- Is the buggy stopped for an object in front of it?

This information is encoded in table 1. Please note that d represents the distance from the buggy to the object in front of it.

Stopping	Right	Left	Signals
False	False	False	0
False	False	True	1
False	True	False	2
False	True	True	3
True	False	False	$4 + 4d$
True	False	True	$5 + 4d$
True	True	False	$6 + 4d$
True	True	True	$7 + 4d$

Table 1. How Signals are Encoded

This system of encoding ensures that information can be compressed without losing any of that information.

4.3 Processing

The control pc uses processing to communicate with the buggy and to create a GUI. The processing sketch can take inputs and display data.

4.3.1 Inputs from user

The ControlP5 is imported in order to take inputs from the user. From this library, a start button, a stop button, a toggle, and a slider are used. The start button sets the variable Motoron to 1, while the stop button sets it to 0. The toggle switches the mode between the velocity input mode and the follow object mode. The slider adjusts the velocity to send to the buggy if the buggy is set to velocity input mode.

4.3.2 Data Transmission

The buggy and the control PC send messages back and forth as outlined previously. If the mode is set to the input velocity mode the message to the Arduino is sent with the line:

```
myClient.write((Motoron*(1*p5.getController("SPEED").getValue())) + "\n");
```

and if the mode is set to the follow object mode the message is sent by the line:

```
myClient.write(Motoron*-1 + "\n");
```

The significance of these messages has already been outlined so we will move on to the messages received from the buggy now.

The Arduino sends a message composed of 3 parts as outlined previously, consisting of distance, voltage, and signals. This data is converted to a list of length 3, split by spaces. Distance can be converted to an integer and taken as is, and voltage can with similar ease be converted to a int and divided by 51 to convert from a 0-255 scale to a 0-5V scale. Signals is slightly more complex, and requires the use of the floor division and modulo functions to parse.

`Left = int(datalist[1])%2!=0;`

`Right = int(datalist[1])%4>=2;`

`Obstruction = int(datalist[1])/4 - 1;` This means Obstruction will equal -1 if there is nothing within 10cm and will otherwise equal the distance to the object in front of it.

4.3.3 GUI

The graphics are technically insignificant but help to monitor the buggy. A brief summary of the different components of the display are outlined below with a supplementary image:

- VelocityMeter: Using the Meter library, this meter displays velocity as calculated from subsequent distance readings from the buggy.
- VoltageMeter: This meter displays the voltage received from the buggy.
- signal(int): This function draws a turn signal for left or right (depending on the integer given, either 0 or 1). The signal will be dark blue if the buggy is not turning in that direction and bright green if it is.
- stopsignal(): Draws an octagon, in red if the buggy is stopped for an impeding object and in dark blue otherwise.
- distanceBox(): Draws a box and displays the total distance travelled so far as sent by the buggy.
- updateGraph(float): Draws a time velocity graph using velocity readings over the past few seconds.

The GUI used for the silver challenge is shown in figure 6.

4.4 PID

The PID settings were honed through a combination of theoretical and experimental tuning. Roughly every 0.4 seconds, the PID function is called with parameters of target velocity v_{target} and measured velocity $v_{measured}$. The error, the integral of the error and the derivative of the error were used to find and maintain a suitable voltage according to the following formula:

$$E(t) = v_{target} - v_{measured}$$

$$V_{output} = k_e E(t) + k_i \int_0^t E(t) dt + k_d \frac{dE(t)}{dt}$$

When this function is discretized it looks more like this:

$$E_i = v_{target} - \frac{\Delta d}{\Delta t}$$

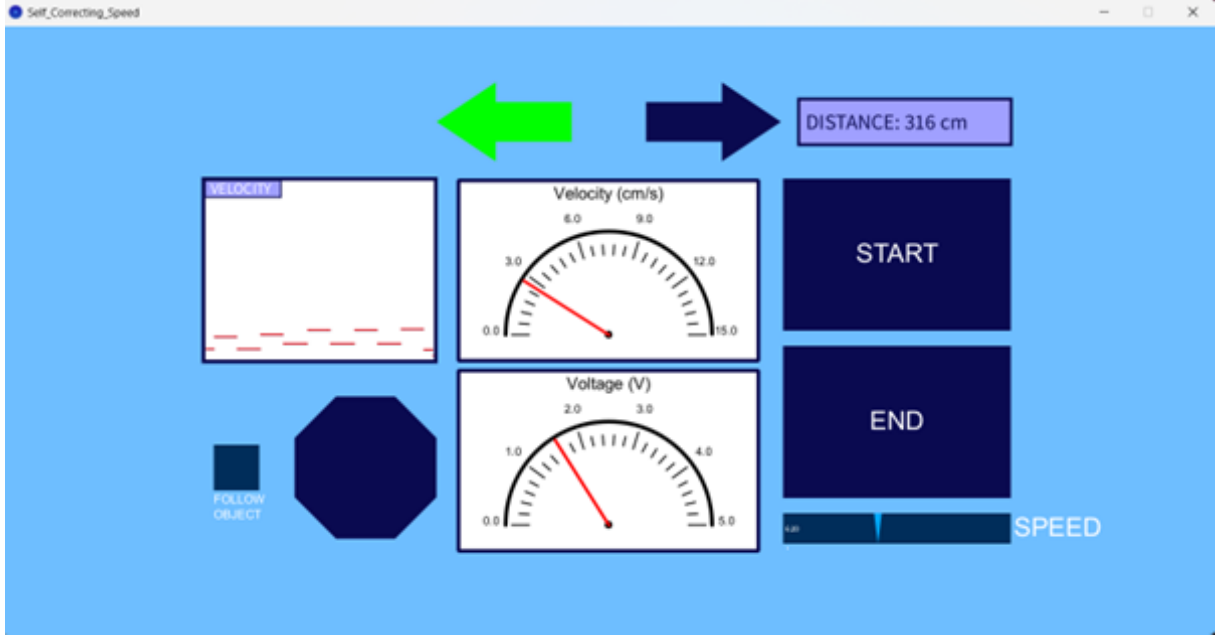


Figure 6. Silver Challenge GUI

$$V_{output} = k_e E_i + k_i \sum_i E_i + k_d \frac{\Delta E}{\Delta t}$$

One of the key adjustments made was in the way speed is measured. If the interval over which speed is measured is too short, the speed measurements are subject to large fluctuations, while if the intervals are too large, the updates to the system are not frequent enough. The way that this dilemma was handled was through the use of floating averages. In short, the measured velocity inputted into the PID would be an average of 4 velocity readings over the last 1.6 seconds. This way, the velocity would update frequently without being subject to significant fluctuations in velocity.

$$v_{measured} = \frac{v_0 + v_1 + v_2 + v_3}{4}$$

This rolling average method of measuring velocity meant that the velocity measurement would not change rapidly, even when the velocity would change quickly. To amend this effect on the PID, the coefficient of the derivative was increased to better account for the change in velocity.

4.5 Object Following

Object following was about following an object in front of the buggy at a safe distance of 15cm. It used ultrasonic sensors to detect how far it was from objects in front of it and how fast objects in front of it were traveling. In order to maintain the right distance and velocity, both the velocity of the target in front and the distance to the buggy. To set the target velocity, the formula below was used to account for both of these targets.

$$v_{target} = k_e(d_{measured} - d_{target}) + k_i \int_0^t v_{object} - v_{measured} dt$$

Object following relied on significant discarding of errors in order to be effective. This is because the ultrasonic sensor will sometimes not be able to detect objects right in front of

it. To tackle this issue, a system of error ignoring was implemented, one that would ignore drastic changes in distance readings without impeding the buggy's ability to detect immediate obstacles in its path. The line of code is

```
if ((dold>=10&&dnw>=10&&abs(dnew-dold)>=10)||((dnw>50)) {dnw=dold;}
```

This code has multiple effects: It will not accept any new distance value which differs from the previous value by 10 or more, unless either the old value or the new value should cause the buggy to stop entirely. This is to ensure that it retains its ability to stop and go when an object is placed in or removed from its immediate path. The other effect the code has is that it does not accept distance values greater than 50cm. This is helpful as it means that the buggy will not follow until an object is placed immediately in front of it.

5 Part D: Gold Challenge

5.1 Overview

The gold challenge involved making several upgrades to the buggy software to allow the Arduino to process the inputs from a HUSKYLENS camera. That being said, most of the program is identical to that previously outlined for the silver challenge, so the parts being outlined here are mainly the parts that changed from silver to gold.

5.2 HuskyLens Code

The HuskyLens camera recognises and processes 5 tags, creatively named ID1, ID2, ID3, ID4, and ID5. They each have associated functions for the Arduino to carry out:

- ID1: Turn left at next junction. Set the variable leftTurn to true. When the IR sensors detect white lines beneath both the left and right sensors, the buggy realises it is at a junction, and will turn left.
- ID2: Turn right at next junction. Set the variable leftTurn to false. When the IR sensors detect white lines beneath both the left and right sensors, the buggy realises it is at a junction, and will turn right.
- ID3: Obey a speed limit. Set the variable speedLimit to true and maxSpeed to false. The program will not allow the speed input v to exceed the pre-set speed limit VLIM.
- ID4: Go at the maximum safe speed. Set the variable speedLimit to false and maxSpeed to true. The program will overrule all other inputs for target velocity and set v to be maximum velocity VMAX.
- ID5: Reset speed settings. Set both variables speedLimit and maxSpeed to false, so that the buggy reverts to doing whatever the control PC tells it to do.

5.3 Data Transmission

With the introduction of new information to be transmitted, the group set out to compress the size of the data to be transmitted between the Arduino and the Control PC. In doing this, the message was reduced from 3 integers separated by spaces to 2 integers separated by spaces. This is how the encoding works:

	Distance	Voltage
Range	0 – ∞	0 – 255
Example Number	107	203
Multiplier	256	1
Number	27392	203
Message (Decimal)	27595	
Message (Hexadecimal)	6BCB	

Table 2. How telemetry is encoded

	Obstruct	maxSpeed	speedLimit	leftTurn	Right	Left
Integer Range	0 – 11	0 – 1	0 – 1	0 – 1	0 – 1	0 – 1
Example Value	7	1	0	0	1	0
Multiplier	32	16	8	4	2	1
Number	224	16	0	0	2	0
Message (Decimal)	242					
Message (Binary)	1111 0010					

Table 3. How signals is encoded

- The first data point is called telemetry. It consists of 2 components: voltage and distance. Distance can be any integer value, but voltage cannot be greater than 255. So if we multiply distance by 256 and add voltage, we can get a unique value of telemetry for every distance and voltage value. This encoding can be thought of as a conversion to hexadecimal where the last 2 digits represent the voltage and all other digits represent distance. An example of this encoding is shown in table 2.
- The second data point is called signals. It consists of 6 separate pieces of information: turning left, turning right, turn left or right at next junction, obeying speed limit, going at maximum safe speed, and being obstructed (and how far is the obstruction). Importantly, all bar 1 of these pieces of information are a simple true or false, so the information can be easily converted to a binary signal. An example of how this works is shown in table 3.

5.4 Updates to Processing

5.4.1 Decoding

There are 2 values to decode from the Arduino, and 8 pieces of information must be extracted from them. This is a brief overview of that process:

First value: Distance and Voltage: If this number is thought of as a hexadecimal value, it can be interpreted as the smallest 2 digits representing voltage and all larger digits representing distance. The specific code is as follows (Note that 'datalist[0]' is the number value in question):

`distance = int(datalist[0])/256;` Integer division disregards voltage values

`Voltage = int(datalist[0])%256 / 51.0;` Modulo disregards distance values

Second value: Left, Right, leftTurn, speedLimit, maxSpeed, Obstruction: This can be interpreted as a binary signal, with the first 5 values correlating to the smallest binary digits

and Obstruction being represented by all larger digits. The decoding generally works as follows:

Left = $\text{signals} \% 2 \neq 0$;

Right = $\text{signals} \% 4 \geq 2$;

leftTurn = $\text{signals} \% 8 \geq 4$;

speedLimit = $\text{signals} \% 16 \geq 8$;

maxSpeed = $\text{signals} \% 32 \geq 16$;

Obstruction = $\text{signals} / 32 - 1$;

(Please note that while this is indicative of how these values are assigned, some of them are not written exactly like this in the code)

5.4.2 GUI

All of the previous elements of the GUI from the silver challenge were retained, but several new components were added to the GUI to reflect the new functionalities of the buggy. A snapshot from this GUI is shown in figure 7.

- Speed Limit Indicator: Displays the speed limit that the buggy is obeying.
- Next Junction Turn Indicator: Displays whether the buggy will turn left or right at the next junction.
- Driving Mode Indicator: Shows whether the buggy is using an input speed or following an object, and whether or not it is in the speed limit mode and the max speed mode.
- Instructions List: Shows a list of the last 15 instructions taken from tags.
- Tag Display: Shows the last tag read by the huskylens.



Figure 7. Gold Challenge GUI

6 Part E: Change Log

Github was not used for this project, but code was saved at various key points during the project. These are attached and the changes are described below:

- **BRONZE_CHALLENGE_FINAL**: Implements the basic functionality of the bronze challenge, such as line following and stopping for objects.
- **SILVER_CHALLENGE_V1**: Implements a basic PID system. Now transmits voltage to processing and receives an input velocity.
- **SILVER_CHALLENGE_V2**: Implements a basic object following program. Control PC can tell it to follow an object in front of it.
- **SILVER_CHALLENGE_V3**: Implements stability measures for object following. Throws away some bad readings and tries to maintain 15cm distance.
- **SILVER_CHALLENGE_FINAL**: Implements stability measures for object following and normal input velocity programs. Now measures velocity with a rolling average. Also reconfigured messaging system to control pc.
- **GOLD_CHALLENGE_FAIL**: Contains some basic software for the tag recognition. Failed due to reasons outlined in hardware challenges section.
- **GOLD_CHALLENGE_V1**: Implements code necessary to recognize tags and carry out respective actions. Most of the subsequent changes after this are aesthetic. Also shows in processing whether buggy is obeying a speed limit and whether the buggy will turn left or right at the next junction.
- **GOLD_CHALLENGE_V2**: Implements box which shows what driving modes the buggy is obeying.
- **GOLD_CHALLENGE_FINAL**: Compresses messaging system to one previously described between buggy and control pc. Also implements a list of the most recent instructions and shows an image of the last tag shown.

7 Conclusion

To conclude, the team worked very well together, and was consistently able to deliver work on time, as well as work to solve problems encountered during the project. While the group was not fully able to complete the Gold challenge, important lessons were learned and good progress was made.